

Data Structures and Algorithms

Assignment 6 : Binary Decision Diagrams (BDD)

Vasiliki Zerva

May 2025

*Faculty of Informatics and Information Technology – Slovak
University of Technology in Bratislava (FIIT – STUBA)*

Class BDD Node

```

7   class BDDNode:
8       def __init__(self, var=None, low=None, high=None, value=None):
9           self.var = var
10          self.low = low
11          self.high = high
12          self.value = value

```

The 1st thing that is important to include is the class of each BDD node. There is the variable name, a low child with variable 0, a high child with variable 1 and a terminal value that can take either 0 or 1.

Main BDD Structure

```

14  class BDD:
15      def __init__(self, root, variables, size):
16          self.root = root
17          self.variables = variables
18          self.num_vars = len(variables)
19          self.size = size

```

Then the class for the whole diagram is included.

Evaluation Function

```

21  def eval_function(expr, var_assignments):
22      try:
23          return str(int(eval(expr, {}, var_assignments)))
24      except:
25          return None

```

Evaluation function is essential as it evaluates the boolean expression using a dictionary of variable assignments. So, it replaces variable names with their values and it calculates the result.

Build BDD Function

```

27 def build_bdd(expr, variables, index, memo, assignment):
28     if index == len(variables):
29         result = eval_function(expr, assignment)
30         return BDDNode(value=result)
31
32     var = variables[index]
33     key = (index, tuple(sorted(assignment.items())))
34     if key in memo:
35         return memo[key]
36
37     assignment[var] = 0
38     low = build_bdd(expr, variables, index + 1, memo, assignment.copy())
39
40     assignment[var] = 1
41     high = build_bdd(expr, variables, index + 1, memo, assignment.copy())
42
43     if low.value == high.value and low.value is not None:
44         return low
45
46     node = BDDNode(var=var, low=low, high=high)
47     memo[key] = node
48     return node

```

Build BDD Function's purpose is to build a complete BDD tree based on the given Boolean function and variable order. At each recursion level, it picks the next variable and it uses a memo dictionary to share identical subtrees, so the BDD gets reduced. It assigns 0 for left and 1 for right and builds both subtrees. At the end, it reduces nodes where both branches lead to the same result and it returns the root node of the tree.

Count Nodes Function

```

50 def count_nodes(node, visited):
51     if node in visited:
52         return 0
53     visited.add(node)
54     if node.value is not None:
55         return 1
56     return 1 + count_nodes(node.low, visited) + count_nodes(node.high, visited)

```

This function counts the number of the unique nodes in a BDD, so the reduction percentag can be calculated.

BDD Create Function

```
58 def BDD_create(boolean_function, variable_order):
59     memo = {}
60     assignment = {}
61     root = build_bdd(boolean_function, variable_order, 0, memo, assignment)
62     size = count_nodes(root, set())
63     return BDD(root, variable_order, size)
```

BDD_create creates a reduced BDD using a fixed variable order. It converts a variable order into a list and it calls the build_bdd function to build the reduced tree. Then it counts the nodes of the final tree and it returns a BDD object with root and size.

Generate Orders Function

```
65 def generate_orders(variables):
66     orders = []
67     n = len(variables)
68     for i in range(n):
69         orders.append(variables[i:] + variables[:i])
70     return orders
```

The generate_orders function has as a purpose to generate different orderings of variables, which influence the structure and size of the BDD.

BDD Create with Best Order

```

71
72 def BDD_create_with_best_order(boolean_function):
73     variables = sorted(list({c for c in boolean_function if c.isalpha()}))
74     orders = generate_orders(variables)
75
76     best_bdd = None
77     best_size = float('inf')
78
79     for order in orders:
80         bdd = BDD_create(boolean_function, order)
81         if bdd.size < best_size:
82             best_size = bdd.size
83             best_bdd = bdd
84
85     return best_bdd

```

This function has as a purpose to find the best variable order, so it can build the smallest possible BDD. So, for it to work, it extracts variables from the Boolean expression and it generates some unique permutations of variable order. It builds a BDD for each order using the function `BDD_create` and it picks the one with the fewest nodes. As a result, it gives the best BDD.

BDD Use Function

```

87 def BDD_use(bdd, input_string):
88     if len(input_string) != len(bdd.variables):
89         return -1
90
91     current = bdd.root
92     var_index = {v: i for i, v in enumerate(bdd.variables)}
93
94     while current.value is None:
95         var = current.var
96         idx = var_index[var]
97         if input_string[idx] == '0':
98             current = current.low
99         elif input_string[idx] == '1':
100             current = current.high
101         else:
102             return -1
103
104     return current.value

```

With this function, the evaluation of specific input values using a BDD is possible. It starts from the BDD root and at each node it reads the corresponding bit from input and follows the low or high branch. It stops at a terminal node and returns its value.

Generate Random Expression Function

```

106 def generate_random_expression(variables, num_clauses=3):
107     expr = []
108     for _ in range(num_clauses):
109         clause = []
110         for var in variables:
111             if random.choice([True, False]):
112                 if random.choice([True, False]):
113                     clause.append(f"not {var}")
114                 else:
115                     clause.append(f"{var}")
116         if clause:
117             expr.append("(" + " and ".join(clause) + ")")
118     return " or ".join(expr) if expr else "0"

```

It is useful to create random Boolean functions for testing the BDD implementation. It takes a list of variable names and it

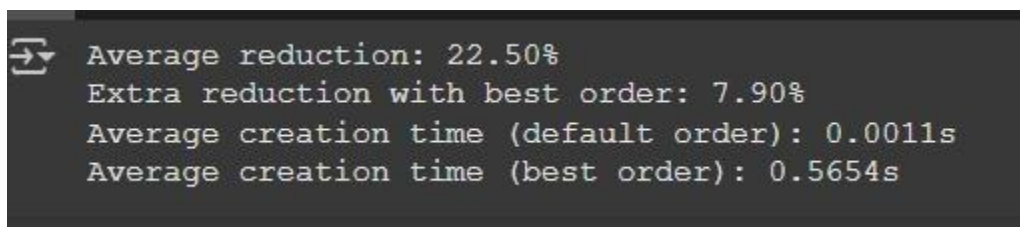
randomly generates a Boolean expression in Disjunctive Normal Form (DNF).

Test BDD System Function

```
120 def test_bdd_system(num_vars=5, num_tests=100):
121     variables = [chr(65 + i) for i in range(num_vars)]
122     total_reduction = 0
123     total_extra_reduction = 0
124     total_time_create = 0
125     total_time_best = 0
126
127     for _ in range(num_tests):
128         expr = generate_random_expression(variables)
129
130         start_time = time.time()
131         bdd1 = BDD_create(expr, variables)
132         end_time = time.time()
133         total_time_create += end_time - start_time
134
135         start_time = time.time()
136         bdd2 = BDD_create_with_best_order(expr)
137         end_time = time.time()
138         total_time_best += end_time - start_time
139
140         max_nodes = 2 ** num_vars
141         reduction = (1 - bdd1.size / max_nodes) * 100
142         extra_reduction = (1 - bdd2.size / bdd1.size) * 100
143
144         total_reduction += reduction
145         total_extra_reduction += extra_reduction
146
147         for bits in itertools.product([0, 1], repeat=num_vars):
148             input_str = ''.join(map(str, bits))
149             var_dict = {variables[i]: bits[i] for i in range(num_vars)}
150             expected = eval_function(expr, var_dict)
151             got = BDD_use(bdd1, input_str)
152             if expected != got:
153
154                 print("Mismatch:", expr, input_str, expected, got)
155
156         print(f"Average reduction: {total_reduction / num_tests:.2f}%")
157         print(f"Extra reduction with best order: {total_extra_reduction / num_tests:.2f}%")
158         print(f"Average creation time (default order): {total_time_create / num_tests:.4f}s")
159         print(f"Average creation time (best order): {total_time_best / num_tests:.4f}s")
160     test_bdd_system(num_vars=13, num_tests=100)
```

Here is the function that implements all of the testing. It generates random Boolean functions and for each one it builds BDD using `BDD_create`, it builds BDD using `BDD_create_with_best_order`, it validates all possible input combinations using `BDD_use` and it does some measurements. The measurements are about the correctness, the percentage of reduction of the tree, the extra reduction from best order and the execution time. Lastly, it prints the results of the measurements.

Measurements – Results

A terminal window with a dark background and light green text. It shows four lines of output: 'Average reduction: 22.50%', 'Extra reduction with best order: 7.90%', 'Average creation time (default order): 0.0011s', and 'Average creation time (best order): 0.5654s'.

```
➔ Average reduction: 22.50%  
Extra reduction with best order: 7.90%  
Average creation time (default order): 0.0011s  
Average creation time (best order): 0.5654s
```

This is the result of an example of `num_vars=5` and `num_tests=10`. The testing could not be done properly with `num_vars=13` and `num_tests=100`, as it was required. Because of the many data and many testings, the execution could not load till the end.

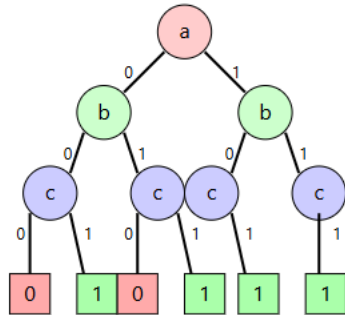
Description of the algorithm with a diagram

BDD Construction and Reduction Process

Step 1: Boolean Function

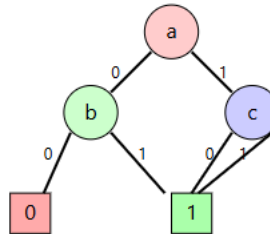
$$f(a,b,c) = a \cdot b + !a \cdot c$$

Step 2: Complete Decision Tree



Step 3: Reduced BDD

Reduce



Variable Ordering Impact

Order: $a \rightarrow b \rightarrow c$

5 nodes

Order: $b \rightarrow a \rightarrow c$

7 nodes

BDD Usage Example

